Software Engineering Lab Experiment - 7

COCOMO Cost Driver Analysis — CustomerChurnPredictor (INTN2333 Case Study)

What COCOMO Is

COCOMO (Constructive Cost Model) is a software cost estimation model used to estimate the **effort**, **schedule**, and (optionally) **cost** required to develop a software project. It works by combining:

1. **Project size** (commonly in SLOC/KLOC)
2. **Scale drivers** (factors that affect how effort grows as the project grows)
3. **Cost drivers** (effort multipliers such as complexity, documentation needs, tool support, schedule pressure, and team distribution)

COCOMO II (the modern version) produces outputs such as:

- **Effort** measured in **person-months**
- **Schedule** measured in **months**
- **EAF (Effort Adjustment Factor)**, which shows how much the cost drivers increased/reduced effort compared to a nominal baseline

In short, COCOMO helps you **identify the main "cost drivers"** (the inputs that push effort up or down) and quantify their impact.

Tools Used to Perform COCOMO on the Team Project

For CustomerChurnPredictor, you used two practical tools:

1) Local SLOC measurement in the project repository (Terminal + PowerShell)

To use COCOMO accurately, you first need an estimate of **project size**. You attempted to use a `cloc` package, but since it didn't reliably produce a CLI command, you used a **PowerShell-based counting approach** that:

- recursively scanned your project directory,
- excluded common non-source folders (like `.git`, `node_modules`, virtual environments),
- counted **non-empty** lines in typical source file extensions (Python/SQL/JS/HTML/CSS/YAML).

This produced a reliable **TOTAL_SLOC_APPROX** and **KSLOC** value.

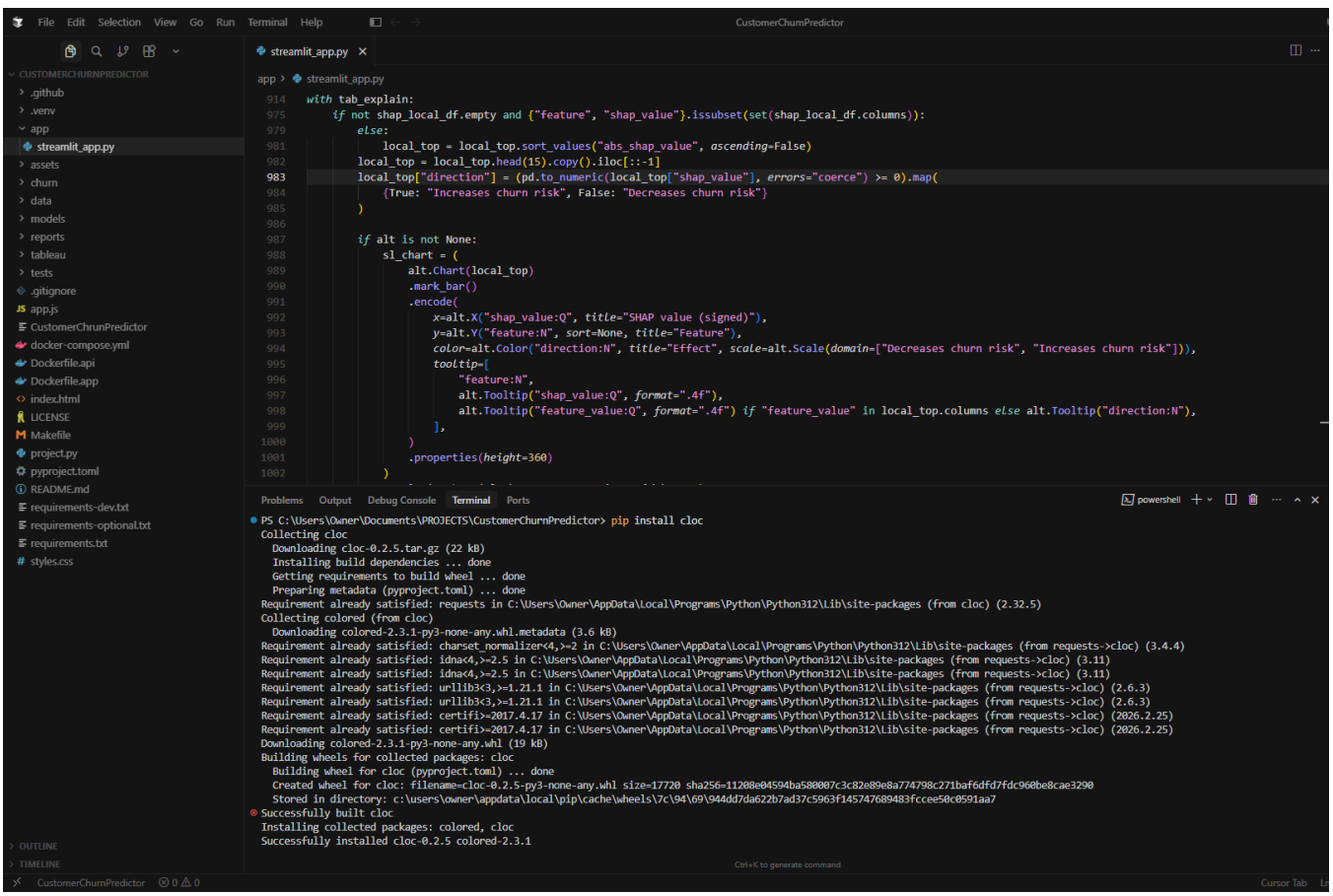
2) The COCOMO II web calculator ([SoftwareCost.org tool](#))

You then used the **COCOMO II site** to input:

- your measured **SLOC**
- your selected scale drivers and cost drivers (based on the nature of your project)
- and generated effort/schedule outputs plus phase distribution.

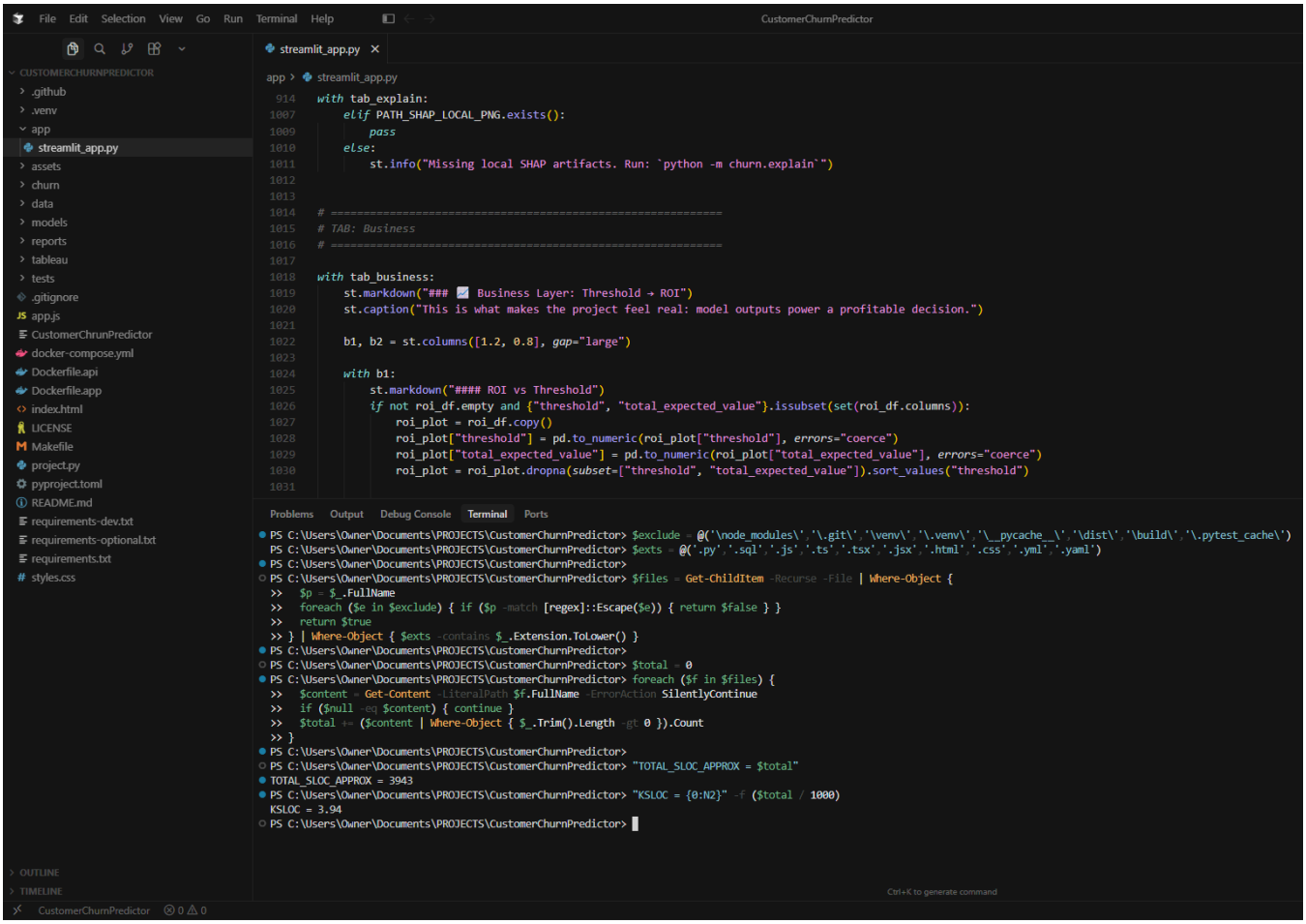
Procedure (Step-by-Step) Based on Your 4 Screenshots

Step 1: Environment setup in the project terminal (VS Code)



In your VS Code terminal, you first confirmed you were working inside the **CustomerChurnPredictor** project directory and attempted to set up a code line counter tool (`pip install cloc`). This step shows the correct workflow mindset: you tried to use an automated tool to measure software size before estimating cost. Even though `cloc` didn't work as expected as a CLI tool in your environment, this step still demonstrates that you followed an industry-style approach: **estimate size first**, then estimate cost.

Step 2: Final SLOC measurement using PowerShell (successful)



You then used a PowerShell script to:

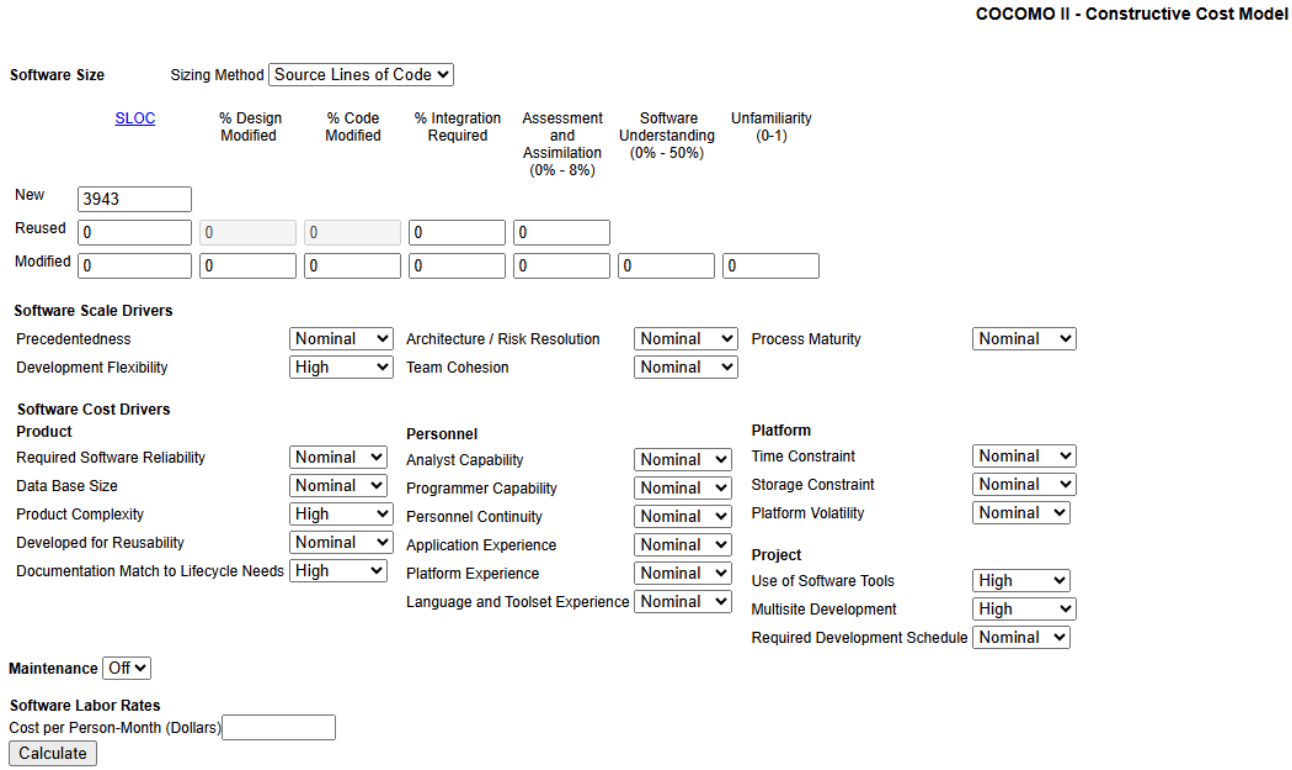
- exclude irrelevant folders (`node_modules`, `.git`, `venv`, etc.)
- restrict counting to relevant source extensions (`.py`, `.sql`, `.js`, `.html`, etc.)
- count **non-empty lines**, which is a good approximation for true SLOC

Your terminal output confirms:

- **TOTAL_SLOC_APPROX = 3943**
- **KSLOC = 3.94**

This is the key input required by COCOMO II, because size is the baseline driver for effort.

Step 3: Entering inputs into the COCOMO II calculator



On the COCOMO II page, you selected:

- **Sizing Method:** Source Lines of Code
- **New SLOC: 3943**
- Reused/Modified: **0** (meaning this is treated as mostly new code)

Then you selected the scale and cost drivers to match a team ML + dashboard project:

Scale drivers (how effort scales):

- Precedentedness: Nominal
- Development Flexibility: **High**
- Architecture/Risk Resolution: Nominal
- Team Cohesion: Nominal
- Process Maturity: Nominal

Cost drivers (effort multipliers):

- Product Complexity: **High**
- Documentation Match to Lifecycle Needs: **High**
- Use of Software Tools: **High**
- Multisite Development: **High**
- Most other drivers: Nominal

These choices are consistent with CustomerChurnPredictor because it's not "just a script." It includes multiple components (data prep + modelling + dashboards + explainability + reporting), has case-study documentation expectations, relies heavily on tools/frameworks, and is built by a distributed team.

Step 4: Capturing COCOMO outputs and phase distribution

Results

Software Development (Elaboration and Construction) Staffing Profile

Effort = 14.2 Person-months
Schedule = 8.5 Months
Cost = $0

Total Equivalent Size = 3943 SLOC
Effort Adjustment Factor (EAF) = 1.09

Acquisition Phase Distribution

Phase	Effort (Person-months)	Schedule (Months)	Average Staff	Cost (Dollars)
Inception	0.9	1.1	0.8	$0
Elaboration	3.4	3.2	1.1	$0
Construction	10.8	5.3	2.0	$0
Transition	1.7	1.1	1.6	$0

Software Effort Distribution for RUP/MBASE (Person-Months)

Phase/Activity	Inception	Elaboration	Construction	Transition
Management	0.1	0.4	1.1	0.2
Environment/CM	0.1	0.3	0.5	0.1
Requirements	0.3	0.6	0.9	0.1
Design	0.2	1.2	1.7	0.1
Implementation	0.1	0.4	3.7	0.3
Assessment	0.1	0.3	2.6	0.4
Deployment	0.0	0.1	0.3	0.5

Your output file is at http://softwarecost.org/tools/COCOMO/data/COCOMO_March_22_2026_15_17_18_277337.txt

Created by Ray Madachy at the Naval Postgraduate School. For more information contact him at rjmadach@nps.edu.

After calculating, the Results section produced:

- **Effort: 14.2 person-months**
- **Schedule: 8.5 months**
- **Total Equivalent Size: 3943 SLOC**
- **EAF: 1.09**

It also produced a phase distribution:

- **Inception:** 0.9 PM, 1.1 months
- **Elaboration:** 3.4 PM, 3.2 months
- **Construction:** 10.8 PM, 5.3 months
- **Transition:** 1.7 PM, 1.1 months

This confirms the model expects the majority of work to occur during **Construction**, where implementation + integration usually dominate.

Interpretation of the Results (Deeper Explanation)

1) What “14.2 Person-Months” Means (in a team context)

A **person-month** is one person working full-time for one month. So **14.2 PM** means the total labour required is equivalent to:

- ~14 months of work for 1 full-time person
- ~7 months for 2 full-time people
- ~3.55 months for 4 full-time people (14.2 ÷ 4)

However, student teams rarely work full-time, which is why the **calendar schedule** estimate becomes longer. COCOMO’s schedule output (8.5 months) reflects typical overheads: coordination, integration, testing, iteration, and rework cycles.

2) Why is the schedule 8.5 Months Even Though Effort is 14.2 PM

COCOMO does not assume that adding more people always reduces the timeline linearly. In reality:

- communication overhead increases with team size,
- integration and merge conflicts become more frequent,
- review cycles and testing take time, even with more people.

So the schedule is not “Effort ÷ People.” The model intentionally keeps the schedule realistic because software development has **dependencies and bottlenecks**.

3) What EAF = 1.09 Actually Says

EAF (Effort Adjustment Factor) multiplies the base effort derived from size and scale drivers. Your **EAF = 1.09** means:

> Your selected cost drivers increased the estimated effort by ~9% compared to a nominal baseline project of the same size.

This is important for “cost driver understanding,” because it shows your project isn’t extreme-risk (EAF isn’t 1.5 or 2.0), but it *does* have additional effort due to complexity, documentation expectations, and team distribution.

Identifying the Key Cost Drivers for CustomerChurnPredictor

Cost Driver #1: Product Complexity (High)

CustomerChurnPredictor includes multiple layers that increase engineering effort:

- data preparation and cleaning (ETL-style work),

- model training and evaluation,
- churn probability scoring and threshold logic,
- dashboards and visual outputs,
- explainability components (your code shows SHAP-style logic and business-oriented interpretation).

Complexity increases effort because it increases:

- number of modules and integration points,
- testing surface area,
- edge cases,
- and debugging time.

Cost Driver #2: Documentation Match to Lifecycle Needs (High)

Because this is a case study and portfolio-grade project, documentation isn't optional—it's part of the deliverable. That includes:

- structured write-ups (problem, approach, results),
- system explanation (workflow + pipeline),
- README clarity and reproducibility steps,
- and possibly diagrams or reporting artifacts.

COCOMO treats documentation as a real effort that affects the schedule and workload across phases.

Cost Driver #3: Multisite Development (High)

Multisite (distributed) development increases cost because:

- decisions take longer (async coordination),
- conflict resolution takes longer (merge conflicts, duplicated work),
- integration becomes heavier (people build parts differently),
- review cycles can slow down.

This is a common hidden cost in team projects and is one reason COCOMO predicts a heavy Construction phase.

Cost Driver #4: Use of Software Tools (High) — a Cost Reducer

This is a positive driver (productivity booster). Strong tool support reduces effort by:

- automating builds/tests,
- simplifying collaboration (GitHub),
- accelerating model development (libraries),
- speeding dashboard creation (BI tooling).

So while some drivers increased effort, TOOL helps offset them—contributing to why your EAF is only 1.09 rather than much higher.

Phase Distribution Analysis (Why “Construction” dominates)

COCOMO predicts **Construction = 10.8 PM**, which is the majority of the total effort. That makes sense for your project because:

- this is where the bulk of implementation happens (ETL + modelling + dashboards),
- integration and debugging occur most heavily here,
- and it’s where teams lose time due to differences in assumptions, data handling, and pipeline design.

The **Transition** phase (1.7 PM) represents final steps like packaging, polishing, deployment/readme cleanup, and demo readiness—things students often underestimate, but COCOMO correctly allocates time for.

Got it. Here’s the **final “COCOMO vs Actual” section** you can append to the write-up, updated for **4 people** and **~3 weeks actual duration**, written in a professional lab/industry tone.

COCOMO Estimate vs Actual Project Timeline

Our team consisted of **4 members** (including myself), and the CustomerChurnPredictor project was completed end-to-end in approximately **3 weeks**. In contrast, the COCOMO II estimate for a project size of **3,943 SLOC** produced an effort of **14.2 person-months** and a schedule of **8.5 months** (EAF = **1.09**). At face value, these numbers appear significantly larger than our observed timeline; however, this difference is expected because COCOMO outputs are calibrated for **typical professional software projects** where “schedule months” assume a more complete lifecycle (formal requirements, architecture, reviews, extensive testing, documentation standards, release hardening, and deployment readiness). In a student case study context, many lifecycle activities are reduced or compressed (e.g., lighter governance, smaller stakeholder overhead, reduced operational constraints, and faster decision-making), which can drastically shorten the calendar duration.

To interpret effort in a team context, **14.2 person-months** corresponds to roughly **3.55 person-months per person** for a 4-person team (14.2 ÷ 4). Converting calendar time, **3 weeks is approximately 0.75 months**, meaning our team achieved an output rate substantially faster than the model’s typical expectation. This can happen when (1) strong tool support accelerates development (libraries, BI tools, version control), (2) requirements are flexible and scope is controlled, and (3) the project is not held to production-level constraints such as formal QA, security compliance, performance SLOs, and long-term maintainability requirements. Therefore, rather than treating the COCOMO schedule as a literal prediction of our student timeline, we use it as a **cost-driver analysis tool**: it highlights what would become expensive in a production-grade version of this same system—especially **complexity (CPLX)**, **documentation expectations (DOCU)**, and **team coordination overhead (SITE)**—even if the student implementation can be completed quickly.

Final Conclusion (Lab-ready)

Using a measured project size of **3,943 SLOC (3.94 KSLOC)** and applying COCOMO II drivers matched to a team-based ML + dashboard system, the model estimates **14.2 person-months of effort** and an **8.5-month schedule**, with an **EAF of 1.09,** indicating a modest increase above baseline due to selected cost drivers. The dominant drivers for CustomerChurnPredictor are **high product complexity**, **high documentation expectations**, and **multisite team coordination overhead**, while strong **tool support** helps reduce effort. The model also shows that most effort occurs in the **Construction phase**, aligning with the reality that implementation and integration consume the majority of time in multi-component team projects.